

Web IDE 中使用 Continue 扩展插件作为 AI 工具

- 1. Continue 扩展插件安装方式
- 2. 为 Continue 插件配置本地模型
- 3. 配置 Chat 等功能中使用的默认模型
- 4. Chat 功能以及 Context Providers

- 5. 编辑已有代码
- 6. Action 功能
- 7. 代码自动补全

User Settings

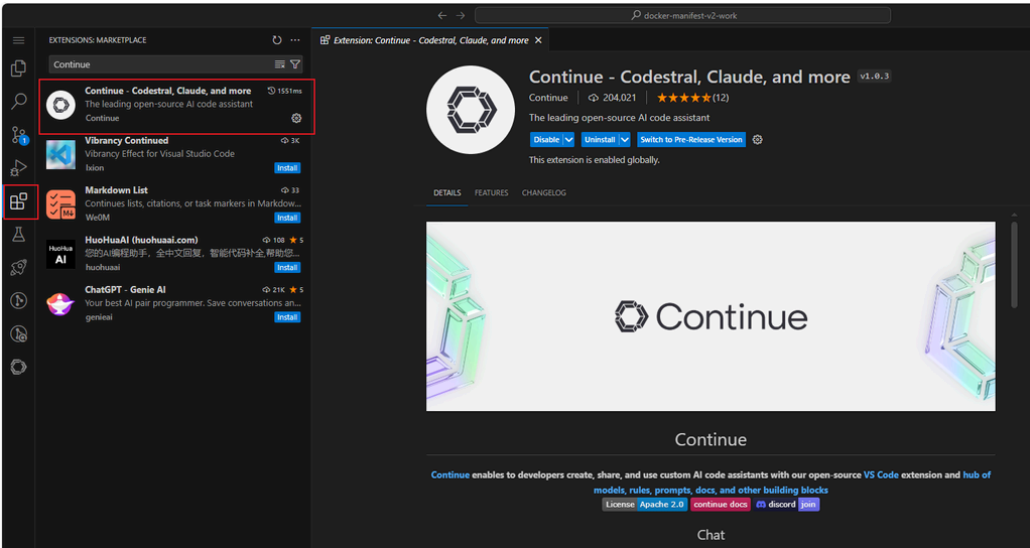
使用不同的provider：

codebase

Prompts Files

1. Continue 扩展插件安装方式

可在 Web IDE 扩展市场 Tab 页，搜索 Continue 插件，并进行在线安装。如下图：



2. 为 Continue 插件配置本地模型

Continue 插件安装好之后，其配置文件位于以下路径：

```
1 ~/.continue/config.yaml
```

当前 Continue 已支持 Ollama、vLLM、OpenAI 等主流的 LLM Provider，可按照其官方文档进行配置。链接如下：

[Model providers | Continue](#)

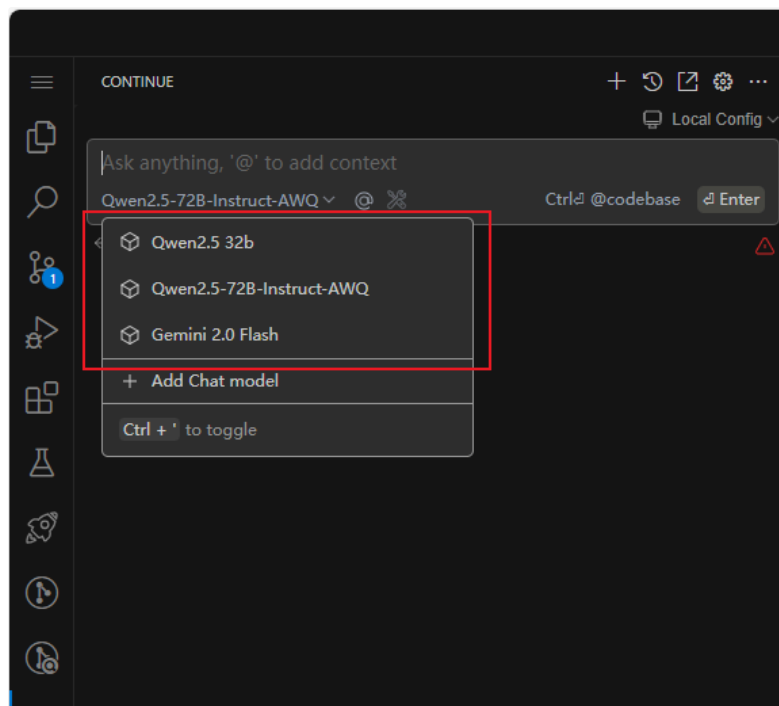
以 vLLM 中的 Qwen2.5-72B-Instruct-AWQ 模型为例，可在 Web IDE 中打开 Terminal 进行配置，内容如下：

```
1 name: Local Assistant
2 version: 1.0.0
3 schema: v1
4 models:
5   - name: Qwen2.5-72B-Instruct-AWQ
```

```
6 provider: vllm
7 model: Qwen2.5-72B-Instruct-AWQ
8 apiKey: token-easystack
9 apiBase: http://172.50.0.3:30080/v1
10 - name: Qwen2.5-Coder-32B-Instruct-AWQ
11 provider: vllm
12 model: Qwen2.5-Coder-32B-Instruct-AWQ
13 apiKey: token-easystack
14 apiBase: http://172.18.0.3:30080/v1
15 roles:
16   - autocomplete
17 context:
18   - provider: code
19   - provider: docs
20   - provider: diff
21   - provider: terminal
22   - provider: problems
23   - provider: folder
24   - provider: codebase
```

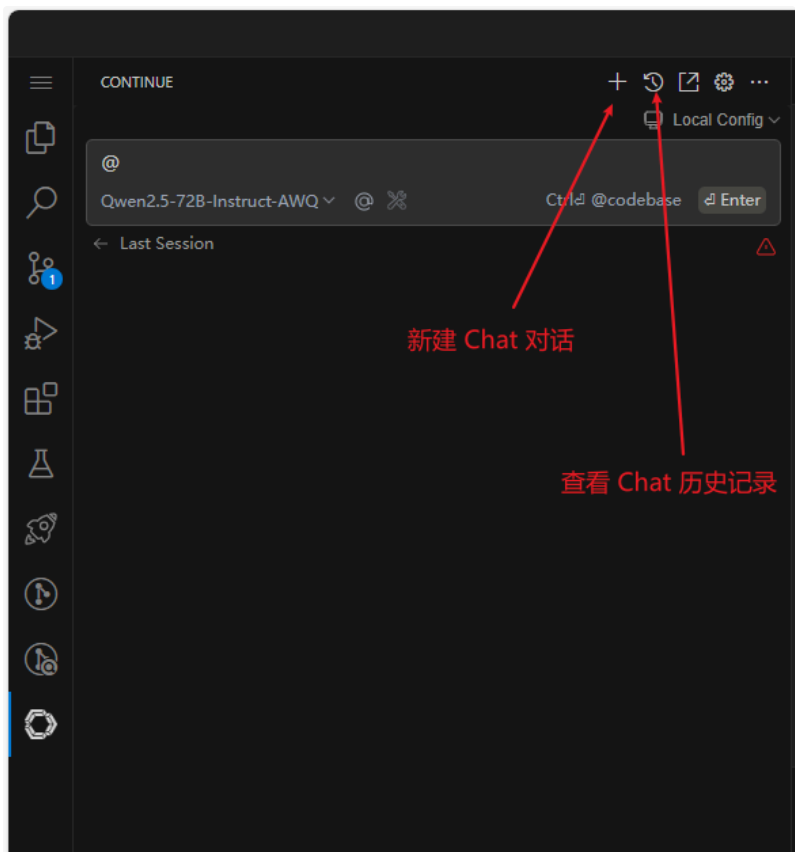
3. 配置 Chat 等功能中使用的默认模型 [🔗](#)

可以在聊天窗口临时切换其他模型。如下图：

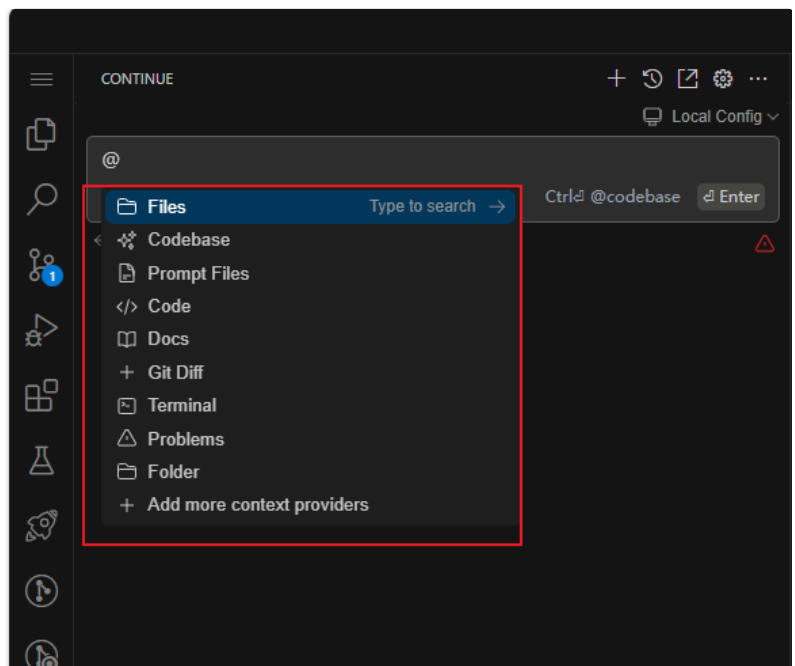


4. Chat 功能以及 Context Providers [🔗](#)

可在 Web IDE 中 Continue 插件的窗口，点击上方的加号，新建 Chat 对话。如下图：



另外 Continue 插件还支持很多的 Context Providers，可在对话框输入“@”符号并选择要使用的 Context Provider。如下图：



以 `Files` Context Provider 为例，指定好文件之后，在聊天时便会将文件内容作为上下文传递给大模型。

以 `Terminal` Context Provider 为例，会将当前 Web IDE 中打开的 Terminal 中的所有命令及执行结果作为上下文传递给大模型。

Continue 目前支持的 Context Providers，官方说明链接如下：

[Context providers | Continue](#)

5. 编辑已有代码 [🔗](#)

可在 IDE 中选中部分代码，然后按 `Ctrl + i` 快捷键。便会将选中的代码带入聊天窗口上下文中，之后输入你对该段代码的要求，大模型处理完成之后，便会直接修改所选中的代码。

Continue 官方说明，链接如下：

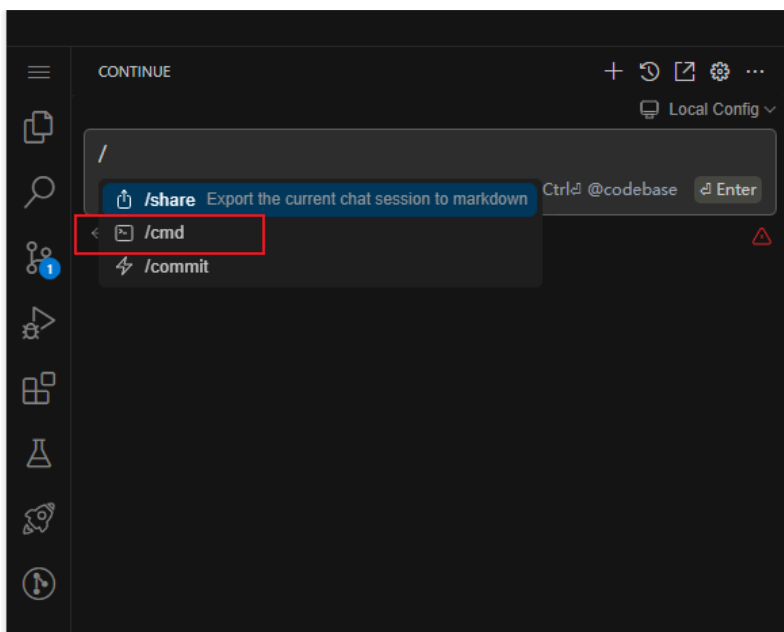
[🔗 Edit | Continue](#)

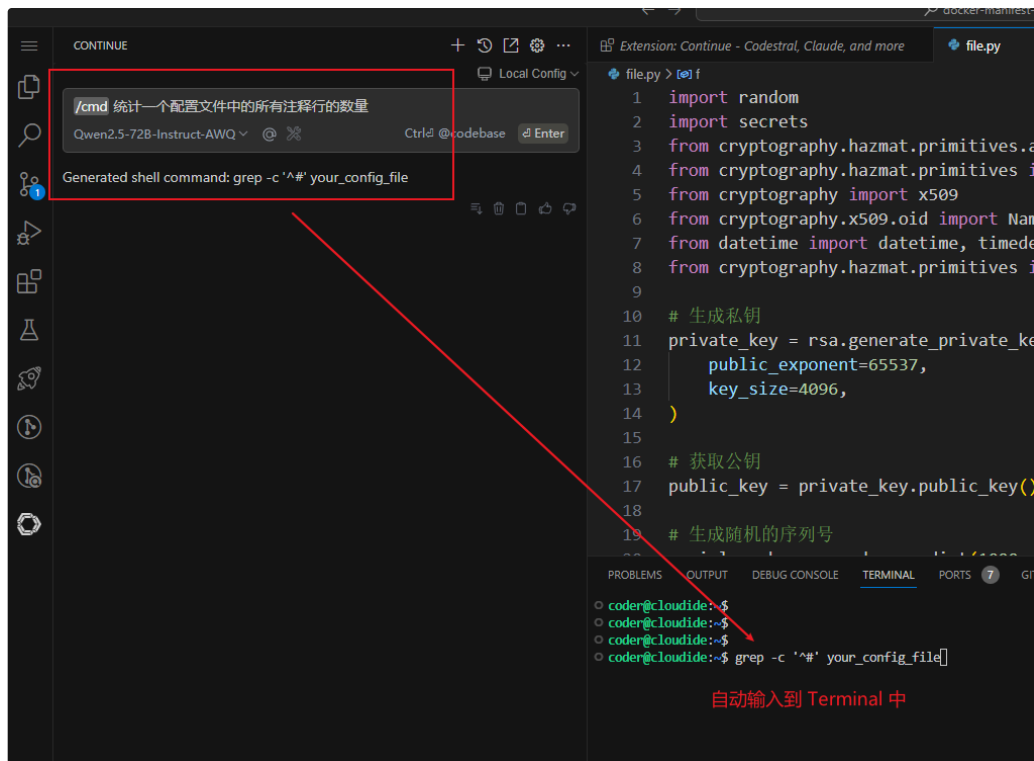
6. Action 功能 [🔗](#)

Action 功能需要在聊天窗口输入 “/” 符号后选择相应的工具进行使用。

以终端命令为例，在聊天窗口输入 “/” 选择 cmd 工具，然后输入需要进行的操作，大模型推理生成答案之后，便会直接应用到 Web IDE 的 Terminal 命令行中。

示例如下图：





Continue 官方文档中关于 Action 功能的说明，链接如下：

[Slash commands | Continue](#)

7. 代码自动补全 [🔗](#)

代码自动补全，即在 IDE 中进行代码开发，大模型可以预测你的下一段代码内容，按 `Tab` 键之后便可以自动补全。

Continue 官方文档中，关于代码自动补全的说明，链接如下：

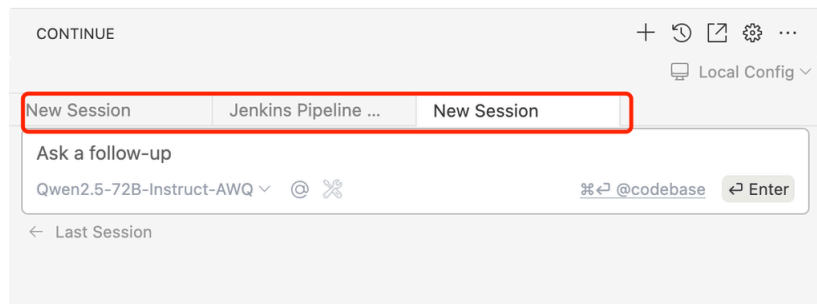
[Autocomplete | Continue](#)

User Settings [🔗](#)

[User Settings Page | Continue](#)

1. Show Session tab

编辑器的continue侧边栏中显示会话标签页，方便在不同AI对话间快速切换，在多任务开发时，保持会话独立



2. Wrap Codeblocks

自动换行长代码块（超过编辑器宽度的代码自动折行显示）

3. Display raw markdown

以原生Markdown格式显示AI响应（而非渲染后的富文本）。前后对比

未开启：



开启后：



4. Show Chat Scrollbar

控制聊天窗口的滚动条显示

5. Allow anonymous telemetry

允许发送匿名使用统计（包括功能使用频率、错误日志），官方描述：

Telemetry

Overview

The open-source Continue Extensions collect and report **anonymous** usage information to help us improve our product. This data enables us to understand user interactions and optimize the user experience effectively. You can opt out of telemetry collection at any time if you prefer not to share your usage information.

We utilize [Posthog](#), an open-source platform for product analytics, to gather and store this data. For transparency, you can review the implementation code [here](#) or read our [official privacy policy](#).

Tracking Policy

All data collected by the open-source Continue extensions is anonymized and stripped of personally identifiable information (PII) before being sent to PostHog. We are committed to maintaining the privacy and security of your data.

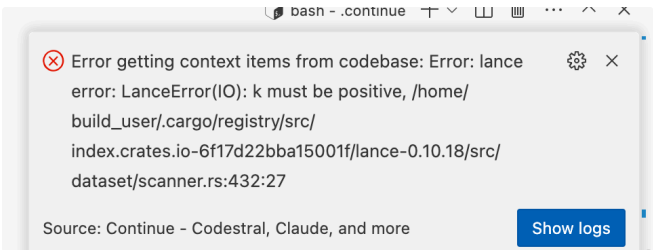
What We Track

The following usage information is collected and reported:

- **Suggestion Interactions:** Whether you accept or reject suggestions (excluding the actual code or prompts involved).
- **Model and Command Information:** The name of the model and command used.
- **Token Metrics:** The number of tokens generated.
- **System Information:** The name of your operating system (OS) and integrated development environment (IDE).
- **Pageviews:** General pageview statistics.

6. Disable indexing

禁用本地代码库的自动索引功能，设置后，将无法使用codebase provider，会出现报错



7. Use autocomplete cache

启用自动补全缓存，将常用补全结果存储至本地，提升下次命中结果的响应时间。

8. Use chromium for docs crawling

使用Chromium内核进行文档爬取，替代默认的轻量级爬虫。

主要是针对docs provider。

9. Response Text to Speech

用TTS朗读LLM生成的响应

10. Disable Session Titles

不自动生成聊天会话的摘要标题

11. Multiline Auto Completions

多行自动补全

- 选项：
 - `always` : 强制多行补全
 - `never` : 仅单行补全
 - `auto` : 根据上下文智能判断 (默认)

12. Disable Autocomplete in Files

禁用文件自动补全, 逗号分隔的glob匹配模式 (如: `*.md, *.txt`)

13. Font Size

字体大小

14. Workspace Prompts Path

自定义Prompt文件存储目录, 默认: `.continue/prompts`

Prompt文件是用于**定制化AI代码生成行为**的核心配置文件。它们通过预设的指令模板和上下文规则, 将自然语言指令转化为精准的代码操作。

使用不同的provider : [🔗](#)

codebase [🔗](#)

使用中出现的**问题**, 在尝试使用@codebase provider分析项目代码时,



会出现报错

此时回答也就不包含code data的上下文。需要添加配置

```
1 {
2   "contextProviders": [
3     {
4       "name": "codebase",
5       "params": {
6         "nRetrieve": 25,
7         "nFinal": 5,
8         "useReranking": true
9       }
10    }
11  ]
12 }
```

配置后的询问结果:

代码中创建jenkins创建的config的流程

Qwen2.5-72B-Instruct-AWQ

@codebase Enter

> 3 context items

在提供的代码片段中，可以看到 `jenkins_utils.py` 文件中的 `__init__` 方法用于初始化 Jenkins 连接对象。虽然这些代码片段没有直接展示创建 Jenkins 项目配置的完整流程，但可以基于这些信息和其他常见的 Jenkins API 操作，来构建一个完整的流程。

1. 初始化 Jenkins 连接

首先，你需要初始化一个 Jenkins 连接对象，这在 `jenkins_utils.py` 文件中已经实现了：

```
jenkins_utils.py Copy Apply

import jenkins

class JenkinsUtils:
    def __init__(self, init_check=True):
        if init_check:
            self.jks_obj = Jenkins(
                baseurl=JENKINS_URL,
                username=DRONE_USER,
                password=DRONE_PASSWORD,
                timeout=JENKINS_RESP_TIMEOUT,
                lazy=True
            )
```

Prompts Files

创建prompt file后，需要刷新页面后，才能引用到。